ScrollGuard Agent Benchmark: Technical Report (v2)

Phase 1 benchmark construction · Generative AI Special Topics · 2026-10-28

1. Summary

I built a benchmark that tests whether ScrollGuard (an intent-aware agent that interrupts mindless scrolling) can read usage context and decide whether to intervene right now.

Current status: five baselines are scored. Running a real LLM agent and independently validating the labels have not been done yet; those two are the main remaining work.

2. How much of the requirements is achieved

Assignment requirements: (a) define the tasks an agent should perform, (b) prepare test cases or evaluation data, (c) establish criteria and metrics for success, and the benchmark must make it possible to evaluate how well an agent achieves the project goals (per the Evaluating LLM lecture).

Requirement	Status	Evidence	Remaining
(a) Task definition	Met	AGENT_SPEC.md specifies the input schema, output JSON, three actions, and label boundary rules	None
(b) Test cases	Mostly met (self-assessed about 75%)	45 cases (18/17/10), 9 adversarial, 8 boundary, four numeric fields taken from real dataset rows, inputs separated from labels, leakage check passed	Independent label validation, realism of synthetic signals, sample size
(c) Success criteria and metrics	Met	Detection rate, false-intervention rate, context-check rate, cost score and more are implemented; five baselines scored	Sensitivity analysis of the cost weights

Requirement	Status	Evidence	Remaining
Ability to evaluate an agent (Evaluating LLM view)	Partial (self-assessed about 40%)	End-to-end pipeline verified with a mock agent	Run a real agent, repeat 3 times for consistency, error analysis

Note: the percentages are our own judgment of work completed against each requirement, not measured values.

3. Benchmark design

3.1 Agent task

Given one usage situation, the agent chooses one of three actions.

- Input (12 columns): time of day, entry trigger, app type, user-stated goal, recent-context text, usage minutes in the last 60 minutes, session count, median session length, usage concentration, swipe-interval coefficient of variation, average dwell per item
- Output: one-line JSON {"action": "...", "reason": "..."}
- Allowed actions: no_intervention / gentle_prompt / ask_context (anything else is invalid)

3.2 Label definitions

Action (intent)	Criterion
no_intervention (intentional)	Behavior matches the stated goal, goal-directed actions are observed (search, save, write, app switching), short utility use, or use within a self-set time budget. Long duration alone does not justify intervening
gentle_prompt (unintended)	Two or more independent autopilot signals and no counter-evidence (no goal-directed action, uniform swipes and short dwell, drifting into the feed after a notification, overshooting a stated goal, timer or bedtime goal, schedule conflict)
ask_context (unclear)	Goal-directed and drifting signals conflict, or key information is missing while the usage is notable. A wrong intervention costs more than a question

Boundary rules: explicit user declarations are respected. Exceeding a time limit alone is insufficient evidence. Very short use (1 to 3 minutes) with no information gets no question.

3.3 Case composition

45 cases in total: no_intervention 18, gentle_prompt 17, ask_context 10. By type: standard 28, adversarial 9 (designed so that time, session count or concentration alone gives the wrong answer), boundary 8. The four usage numbers are real data; time of day, trigger, app type, swipe, dwell and context text are synthetic.

3.4 Metrics and baselines

- Accuracy (with bootstrap 95% CI), macro-F1, confusion matrix
- Detection rate: share of gentle_prompt cases answered correctly
- False-intervention rate: share of no_intervention cases where the agent chose gentle_prompt (central to the product philosophy)
- Unnecessary-question rate and context-check rate: ask_context chosen on no_intervention cases, and ask_context cases answered correctly
- Cost score = 1 minus mean cost (unnecessary intervention 1.0 > miss 0.8 > wrong question 0.4 to 0.6; these weights are a design assumption)
- Accuracy on hard cases (adversarial plus boundary), invalid-output rate, consistency across 3 runs of the same case

- Baselines: B0 always no_intervention / always gentle_prompt / always ask_context; B1 intervene if usage is 30 minutes or more; B2 a rule using only numbers, time and entry trigger, with no text

4. (2) Role of each file

File	Role	Input to output
micro_datasetA.csv	Public raw data (5,777 rows by 17 columns)	Input to build_benchmark.py
inspect_dataset.py, prepare_benchmark_data.py, sample_cases.py	v1 data exploration (kept)	micro_datasetA.csv to console and sample_cases.csv
build_benchmark.py	Defines the 45 cases, draws real rows per tier, writes the files. Replaces v1's create_benchmark_cases.py and split_benchmark.py	micro_datasetA.csv to inputs, labels, meta, annotation_sheet
benchmark_inputs.csv	Inputs given to the agent (no answers)	Read by run_agent.py, check_leakage.py, evaluate.py
benchmark_labels.csv	Ground truth (user_intent, intervention_needed, expected_action); hidden from the agent	Read by evaluate.py and agreement.py
benchmark_meta.csv	Case type, label rationale, source row; hidden	Read by evaluate.py
annotation_sheet.csv	Inputs plus an empty annotator_action column for independent labeling	Filled by annotators, then fed to agreement.py
check_leakage.py	Scans input text for expressions that state the answer	benchmark_inputs.csv to console
run_agent.py	Sends only the inputs to an LLM, parses the JSON, saves predictions. --mock gives random output for pipeline checks	benchmark_inputs.csv to predictions_*.csv

File	Role	Input to output
evaluate.py	Scores the five baselines and any predictions; prints metrics, CI, confusion matrices	inputs, labels, meta, predictions to console and results_summary.csv
agreement.py	Cohen's kappa between annotators and between annotators and the designer	labels and annotator CSVs to console
AGENT_SPEC.md	Specification of task, labels and metrics	Basis for the submitted document
results_summary.csv	Metric table from the last evaluate.py run	Written by evaluate.py

5. Execution flow and call relationships

```
micro_datasetA.csv
      |
      v
build_benchmark.py --> benchmark_inputs.csv --> check_leakage.py (leakage
check)
      |                       |
      |                       v
      |                run_agent.py --> predictions_*.csv --+
      +--> benchmark_labels.csv --------------------------+
      +--> benchmark_meta.csv -----------------------------+--> evaluate.py
--> results_summary.csv
      +--> annotation_sheet.csv --> (independent annotators) --> agreement.py
```

At the function level, the scripts do not import one another; they are connected only through CSV files.

- build_benchmark.py: CASES (45 definitions) then data.query() with the TIERS conditions, then random row selection (seed 42), then four CSV files saved
- run_agent.py: main() then build_user_message(row) then call_llm() then parse() then CSV saved. It never reads the answer files.
- evaluate.py: main() runs the five BASELINES functions and loads prediction files, then evaluate(), score_run() and macro_f1(), then prints the table and saves results_summary.csv

6. How to run, with terminal output

6.1 Setup

Put all files in one folder, including micro_datasetA.csv. In PowerShell, move to that folder and install the packages. The anthropic package is needed only for the real agent run.

```
cd (path of the folder with the files)
pip install pandas numpy anthropic
```

If Korean text appears garbled, run chcp 65001 first.

6.2 Build the benchmark

```
python build_benchmark.py
```

Terminal output (the Korean line means 'Generation complete: 45 cases'):

```
생성 완료: 45 사례
expected_action
no_intervention    18
gentle_prompt      17
ask_context        10
case_type
standard       28
adversarial     9
boundary        8
```

Files created: benchmark_inputs.csv, benchmark_labels.csv, benchmark_meta.csv, annotation_sheet.csv

6.3 Leakage check

```
python check_leakage.py
```

The output means 'Suspected leakage expressions: none':

```
누출 의심 표현: 없음
```

6.4 Score the baselines

```
python evaluate.py --confusion
```

Terminal output as printed, followed by the confusion matrices for B1 and B2 (rows are the true label, columns are the prediction):

```
                         accuracy      acc_ci95 macro_f1 detect_rate
false_intervention question_burden ask_recall hard_acc cost_score
invalid_rate consistency  n_runs
B0_always_no_intervention     0.40  [0.27, 0.53]     0.19        0.00
0.00            0.00       0.00     0.47       0.56         0.00         nan
1
B0_always_gentle_prompt       0.38  [0.24, 0.51]     0.18        1.00
1.00            0.00       0.00     0.24       0.47         0.00         nan
1
B0_always_ask_context         0.22  [0.11, 0.36]     0.12        0.00
0.00            1.00       1.00     0.29       0.65         0.00         nan
1
B1_time_threshold_30min       0.47  [0.33, 0.62]     0.35        0.53
0.33            0.00       0.00     0.24       0.59         0.00         nan
1
B2_feature_rule_no_text       0.58  [0.42, 0.73]     0.43        0.47
0.00            0.00       0.00     0.71       0.71         0.00         nan
1

[B1_time_threshold_30min] 행=정답, 열=예측
col_0            no_intervention  gentle_prompt
expected_action
no_intervention               12              6
gentle_prompt                  8              9
ask_context                    5              5

[B2_feature_rule_no_text] 행=정답, 열=예측
col_0            no_intervention  gentle_prompt
expected_action
no_intervention               18              0
gentle_prompt                  9              8
ask_context                   10              0
```

The metric table is also saved to results_summary.csv.

6.5 Run the agent

Pipeline check (random mock, the scores carry no meaning):

```
python run_agent.py --mock --runs 3 --out predictions_mock.csv
```

The lines mean 'run N done' and 'Saved':

```
run 0 완료
run 1 완료
run 2 완료
저장: predictions_mock.csv
```

Real agent run (an API key is required; replace the model name with the one you use):

```
$env:ANTHROPIC_API_KEY = "your API key"
python run_agent.py --model claude-sonnet-5-5 --runs 3 --out
predictions_agent.csv
```

The output format is the same as the mock run, ending with the line saying predictions_agent.csv was saved. To use another LLM (such as OpenAI), change only the call_llm() function in run_agent.py to that provider's API call. 45 cases times 3 runs makes 135 calls.

6.6 Score the agent

```
python evaluate.py --pred predictions_agent.csv --confusion
```

A predictions_agent row is added below the five baseline rows. For reference, the row produced by the mock file was as follows (it is random, so being below the baselines is expected):

```
predictions_mock              0.29  [0.21, 0.37]     0.27        0.33
0.26           0.41       0.13     0.27       0.56         0.00        0.11
3
```

To compare several agents, list several files after --pred.

6.7 Independent label validation

Give annotation_sheet.csv to two people and have them fill the annotator_action column with one of no_intervention / gentle_prompt / ask_context, then run:

```
python agreement.py annot_a.csv annot_b.csv
```

Output format (the sample below is a sanity check run on a file that copied the answers exactly; real annotators will score lower). The second line means 'Guide: kappa of 0.6 or higher is substantial agreement, below 0.4 means the label criteria need redefinition':

```
annot_a.csv vs 설계자 라벨: 일치 1.00, kappa 1.00
해석 참고: kappa 0.6 이상이면 '상당한 일치', 0.4 미만이면 라벨 기준 재정의 필요
```

With two annotators, an annot_a.csv vs annot_b.csv line is printed as well.

7. Interpreting the current results

- The threshold rule (B1) intervenes unnecessarily in 33% of intentional-use cases. This quantifies the limitation of time-only approaches (context-blindness).
- No baseline can output ask_context (except the always-ask one), so their context-check rate is 0. Context judgment is where an agent can add unique value.
- Agent goal: beat B2 (accuracy 0.58, cost score 0.71) while keeping the false-intervention rate low and getting a context-check rate well above 0.
- Caution: B2 is a rule designed after seeing the cases, so it is an optimistic ceiling. With 45 cases the confidence intervals are wide (B2: 0.42 to 0.73), so small differences should not be interpreted.

8. Limitations and future work

1. Label validity: labels are the designer's expected values. Kappa from two independent annotators is needed before claiming validity.
2. Synthetic signals: time of day, trigger, swipe, dwell and context text are not real observations and are not data collected from users about their intervention preferences.
3. Sample size: 45 is still small. After validation, expanding to about 100 is desirable.
4. Design assumptions: the cost weights and the two-or-more-signals criterion are assumptions. A sensitivity analysis of whether the ranking holds under different weights is needed.
5. Scope of the leakage check: check_leakage.py works at the keyword level and does not detect implicit cues such as writing style or sentence length.
6. Phase 2 operation: to evaluate other teams' agents, keep benchmark_labels.csv and benchmark_meta.csv private, publish the prediction file format (case_id, run, action, reason), and build the leaderboard with evaluate.py --pred.
7. After running the agent, add a one-page error analysis by case_type.